



Propeller Heads Turbine Front End and SDK

Security Assessment

March 4, 2026

Prepared for:

Markus Schmitt

Propeller Heads

Prepared by: **Scott Arciszewski and Tarun Bansal**

Table of Contents

Table of Contents	1
Project Summary	2
Executive Summary	3
Project Goals	5
Project Targets	6
Project Coverage	7
Summary of Findings	8
Detailed Findings	9
1. parseSignature function does not check input length	9
A. Vulnerability Categories	10
B. Code Quality Recommendations	12
C. Known Issues and Planned Improvements	13
D. Fix Review Results	14
Detailed Fix Review Results	15
Status of Known Issues and Planned Improvements	15
E. Fix Review Status Categories	17
About Trail of Bits	18
Notices and Remarks	19

Project Summary

Contact Information

The following project manager was associated with this project:

Tara Goodwin-Ruffus, Project Manager
tara.goodwin-ruffus@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Scott Arciszewski, Consultant
scott.arciszewski@trailofbits.com

Tarun Bansal, Consultant
tarun.bansal@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
December 30, 2025	Pre-project kickoff call
January 21, 2026	Delivery of report draft
January 21, 2026	Report readout meeting
March 2, 2026	Completion of fix review
March 4, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Propeller Heads engaged Trail of Bits to review the security of the Turbine front end and SDK.

Turbine is a privacy-preserving decentralized exchange (DEX) for Ethereum that enables token swaps without revealing users' trading intentions, protecting against MEV attacks such as front-running and sandwich attacks. The scope of this report consists of a TypeScript SDK and React-based web application that allows users to submit swap orders and manage liquidity through an intent-based architecture: users sign orders off-chain using cryptographic signatures, which are then executed by Turbine's back end infrastructure.

A team of two consultants conducted the review from January 5 to January 9, 2026, for a total of two engineer-weeks of effort. Our testing efforts focused on application security risks. With full access to source code and documentation, we conducted static and dynamic testing of the codebases using both automated and manual processes.

Observations and Impact

At the start of the audit, we were provided a list of 13 known issues and planned improvements ([appendix C](#)).

Aside from these known issues, the front end and SDK codebases demonstrate generally sound architecture. We identified only one new informational-severity issue, which is straightforward to remediate with minimal code changes.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Propeller Heads take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Integrate security-focused static analysis into the development pipeline.** A future code change could introduce security risks. We recommend introducing mechanisms that maintain the current security baseline, such as Semgrep and/or CodeQL, in the CI pipeline.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	0
Medium	0
Low	0
Informational	1
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Data Validation	1

Project Goals

The engagement was scoped to provide a security assessment of Propeller Heads' Turbine front end and SDK. Specifically, we sought to answer the following non-exhaustive list of questions:

- How does the "Sign in with Ethereum" (SIWE) implementation handle session life cycle, including creation, validation, and termination?
- Are session identifiers protected against fixation, hijacking, and replay attacks?
- How are cookies and session tokens parsed, stored, and transmitted?
- Does the application use cryptographically secure random number generation for security-sensitive operations (salts, nonces, session IDs)?
- Are cryptographic values (hashes, signatures, addresses) properly validated for format, length, and checksum correctness?
- Does the SDK validate API responses and user inputs before use?
- How does the application handle Ethereum addresses, transaction hashes, and other blockchain-specific data types?
- Are there risks of type confusion or precision loss when converting between numeric representations?
- Are sensitive values (private keys, API credentials) stored and handled appropriately?
- Are there hard-coded values (chain IDs, prices, slippage tolerances) that could lead to incorrect behavior or security issues?

Project Targets

The engagement involved reviewing and testing the following targets.

Propeller Heads Dapp Front End

Repository	https://github.com/propeller-heads/propellerswap-frontend
Version	fba934df4eca0c11f4b84cfaa1860e4142e3a680
Type	TypeScript
Platform	Web

Propeller Heads Turbine SDK

Repository	https://github.com/propeller-heads/turbine-sdk
Version	d5ed0b7615afdf2dcac3faae2eacd02342878960
Type	TypeScript
Platform	Web

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Static and dynamic analysis of the SDK and front end code
- Manual code review by Trail of Bits engineers
- Automated code analysis using **Claude Skills developed by Trail of Bits**

Coverage Limitations

Because testing work is time-boxed, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- Production server configuration (HTTPS, Content-Security-Policy headers, etc.)
- The security of third-party open source software used in Turbine (i.e., React)

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	<code>parseSignature</code> function does not check input length	Data Validation	Informational

Detailed Findings

1. parseSignature function does not check input length

Severity: Informational

Difficulty: Undetermined

Type: Data Validation

Finding ID: TOB-TURBINEFRONT-1

Target: sdk/src/turbineClient.ts

Description

The parseSignature function in the Turbine SDK client assumes, but does not verify, that the input string is 132 hexadecimal characters long.

```
private parseSignature(signature: Hex): any {
  // Parse the 65-byte signature: 32 bytes r + 32 bytes s + 1 byte v
  const r = signature.slice(0, 66); // 0x + 32 bytes
  const s = `0x${signature.slice(66, 130)}`; // 32 bytes
  const v = parseInt(signature.slice(130, 132), 16); // 1 byte

  // Convert v (27/28) to yParity (0/1)
  const yParity = v === 28 ? "0x1" : "0x0";

  return {
    r: r,
    s: s,
    yParity: yParity,
    v: `0x${v.toString(16)}`,
  };
}
```

Figure 1.1: *src/turbineClient.ts#L1199-L1214*

Recommendations

Short term, update the code to check that the signature is the desired length.

Long term, consider adding a fuzz testing workflow in CI that covers every parsing function in the codebase to catch edge cases.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

This appendix contains recommendations for findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Validate external URLs.** The front end renders the `icon_url` from the Solver API without any domain validation. While React prevents XSS via image sources, the front end's lack of validation could allow attackers to embed tracking pixels or inappropriate content if the API is compromised.

C. Known Issues and Planned Improvements

Propeller Heads provided Trail of Bits with a document listing known issues and planned improvements.

- TRB-886: Weak PRNG for salts
- TRB-895: Session ID not cleared on logout
- TRB-902: Binary data as strings
- TRB-901: URL string concatenation
- TRB-900: Missing input validation
- TRB-899: CHAIN_ID hard coded to 1
- TRB-898: Unsafe HTTP requests
- TRB-893: Floating-point precision loss
- TRB-889: Address checksums not verified
- TRB-888: Ad-hoc Cookie header generation
- TRB-887: Incorrect Set-Cookie parsing
- TRB-885: Secrets in environment variables
- TRB-884: Hard-coded ETH price

When we identified the same issues in the code, we did not include them in our findings list.

D. Fix Review Results

During a fix review, Trail of Bits assesses the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not a comprehensive analysis of the system.

On March 2, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Propeller Heads team for the issue identified in this report, as well as the known issues at the time of our audit. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, of the 14 issues described in this report (including 13 known issues, described in [appendix C](#)), Propeller Heads has resolved 13 issues and has not resolved the remaining issue. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	<code>parseSignature</code> function does not check input length	Informational	Resolved

Known Issues and Planned Improvements

ID	Title	Status
TRB-886	Weak PRNG for salts	Resolved
TRB-895	Session ID not cleared on logout	Resolved
TRB-902	Binary data as strings	Resolved
TRB-901	URL string concatenation	Resolved
TRB-900	Missing input validation	Resolved
TRB-899	CHAIN_ID hard coded to 1	Resolved
TRB-898	Unsafe HTTP requests	Resolved
TRB-893	Floating-point precision loss	Resolved

TRB-889	Address checksums not verified	Resolved
TRB-888	Ad-hoc Cookie header generation	Resolved
TRB-887	Incorrect Set-Cookie parsing	Resolved
TRB-885	Secrets in environment variables	Resolved
TRB-884	Hard-coded ETH price	Unresolved

Detailed Fix Review Results

TOB-TURBINEFRONT-1: parseSignature function does not check input length

Resolved in [PR #91](#). The parseSignature function now uses new validation logic that ensures that the input is the required length.

Status of Known Issues and Planned Improvements

TRB-886: Weak PRNG for Salts

Resolved in [PR #88](#). A cryptographically secure random number generator is now used for salts.

TRB-895: Session ID not cleared on logout

Resolved in [PR #448](#) on the Turbine backend repository. Cookies are now cleared on logout.

TRB-902: Binary data as strings

Resolved in [PR #91](#). The new validation logic correctly separates binary data from strings.

TRB-901: URL string concatenation

Resolved in [PR #93](#). The new buildApiUrl function ensures that URLs are valid.

TRB-900: Missing input validation

Resolved in [PR #91](#). Input validation is now a separate component that is imported and used explicitly throughout the SDK.

TRB-899: CHAIN_ID hard coded to 1

Resolved in [PR #93](#). The chain ID is provided by the wallet.

TRB-898: Unsafe HTTP requests

Resolved in [PR #95](#). Response sizes are validated and HTTPS is enforced for non-localhost requests.

TRB-893: Floating-point precision loss

Resolved in [PR #94](#). Values are stored as strings rather than floats, which mitigates the precision issue.

TRB-889: Address checksums not verified

Resolved in [PR #91](#). The `viem isAddress` function is now used to validate addresses and their checksums.

TRB-888: Ad-hoc Cookie header generation

Resolved in [PR #103](#). Cookie headers are now managed with Cookiejar.

TRB-887: Incorrect Set-Cookie parsing

Resolved in [PR #103](#). Cookie headers are now parsed using Cookiejar.

TRB-885: Secrets in environment variables

Resolved in [PR #98](#). Public keys are now loaded from the keystore rather than environment variables.

TRB-884: Hard-coded ETH price

Unresolved as of [PR #107](#). The code has been updated with the following comment:

```
// ⚠ IMPORTANT: Update these amounts and token pairs before running this script!  
// Set realistic amounts based on your needs and current market conditions
```

Figure D.1: [scripts/submit-orders.ts#L42-L43](#)

This does not fundamentally change the hard-coded nature of the ETH price.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Propeller Heads under the terms of the project statement of work and has been made public at Propeller Heads' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.